ADVANCED DATA STRUCTURES & ALGORITHM ANALYSIS LAB

1.Construct an AVL tree for a given set of elements which are stored in a file. And implement insert and delete operation on the constructed tree. Write contents of tree into a new file using inorder.

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1
class AVLTree:
    def get_height(self, root):
        return 0 if not root else root.height
    def get_balance(self, root):
        return 0 if not root else self.get_height(root.left) - self.get_height(root.right)
    def right_rotate(self, y):
        x = y.left
        T2 = x.right
        x.right = y
        y.left = T2
        y.height = 1 + max(self.get_height(y.left), self.get_height(y.right))
        x.height = 1 + max(self.get_height(x.left), self.get_height(x.right))
        return x
    def left_rotate(self, x):
        y = x.right
        T2 = y.left
        y.left = x
        x.right = T2
        x.height = 1 + max(self.get_height(x.left), self.get_height(x.right))
        y.height = 1 + max(self.get_height(y.left), self.get_height(y.right))
        return y
    def insert(self, root, key):
        if not root:
```

```
            return Node(key)
        elif key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root  # duplicate not allowed
        root.height = 1 + max(self.get_height(root.left), self.get_height(root.right))
        balance = self.get_balance(root)
        if balance > 1 and key < root.left.key:
            return self.right_rotate(root)
        if balance < -1 and key > root.right.key:
            return self.left_rotate(root)
        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
        return root
    def min_value_node(self, node):
        current = node
        while current.left:
            current = current.left
        return current
    def delete(self, root, key):
        if not root:
            return root
        elif key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
```

```
            if not root.left:
                return root.right
            elif not root.right:
                return root.left
            temp = self.min_value_node(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)
        root.height = 1 + max(self.get_height(root.left), self.get_height(root.right))
        balance = self.get_balance(root)
        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)
        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)
        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
        return root
    def inorder(self, root, result):
        if root:
            self.inorder(root.left, result)
            result.append(root.key)
            self.inorder(root.right, result)
if __name__ == "__main__":
    avl = AVLTree()
    root = None
    elements = [10, 20, 30, 40, 50, 25]
    print(" ◇  Inserting elements:", elements)
    for num in elements:
        root = avl.insert(root, num)
    insert_val = 15
```

```
print(f" ◇ Inserting element: {insert_val}")
root = avl.insert(root, insert_val)
delete_val = 20
print(f" ◇ Deleting element: {delete_val}")
root = avl.delete(root, delete_val)
inorder_result = []
avl.inorder(root, inorder_result)
with open("output.txt", "w") as f:
    f.write(" ".join(map(str, inorder_result)))
print("\n☑ Final AVL Tree (Inorder Traversal):", inorder_result)
```

output:

◇ Inserting elements: [10, 20, 30, 40, 50, 25]

◇ Inserting element: 15

◇ Deleting element: 20

☑ Final AVL Tree (Inorder Traversal): [10, 15, 25, 30, 40, 50]

2.Construct Min and Max Heap using arrays, delete any element and display the content of the Heap.

```
import heapq
min_heap = []
max_heap = []
elements = [40, 10, 30, 50, 20]
print("Inserting elements:", elements)
for e in elements:
    heapq.heappush(min_heap, e)
    heapq.heappush(max_heap, -e)
print("Min Heap:", min_heap)
print("Max Heap:", [-x for x in max_heap])
val = 30
print(f"\nDeleting {val}...")
if val in min_heap:
    min_heap.remove(val)
    heapq.heapify(min_heap)
if -val in max_heap:
    max_heap.remove(-val)
    heapq.heapify(max_heap)
print("Min Heap after deletion:", min_heap)
print("Max Heap after deletion:", [-x for x in max_heap])
```

output:

```
Inserting elements: [40, 10, 30, 50, 20]
Min Heap: [10, 20, 30, 50, 40]
Max Heap: [50, 40, 30, 10, 20]
Deleting 30...
Min Heap after deletion: [10, 20, 50, 40]
Max Heap after deletion: [50, 40, 10, 20]
```

3. Implement BFT and DFT for given graph, when graph is represented by

a) Adjacency Matrix b) Adjacency Lists

Adjacency Matrix:

```
from collections import deque
graph_matrix = [
    [0, 1, 1, 0],
    [1, 0, 0, 1],
    [1, 0, 0, 1],
    [0, 1, 1, 0]
]
def bft_matrix(graph, start):
    n=len(graph)
    visited = [False] * n
    queue = deque([start])
    visited[start] = True

    while queue:
        v = queue.popleft()
        print(v, end=' ')
        for i in range(n):
            if graph[v][i]== 1 and not visited[i]:
                queue.append(i)
                visited[i] = True

def dft_matrix(graph, start, visited=None):
    if visited is None:
        visited = [False] * len(graph)
    visited[start] = True
    print(start, end=' ')
    for i in range(len(graph)):
        if graph[start][i]== 1 and not visited[i]:
```

```
            dft_matrix(graph, i, visited)

print("BFT (Adjacency Matrix):")
bft_matrix(graph_matrix, 0)
print("\nDFT (Adjacency Matrix):")
dft_matrix(graph_matrix, 0)
```

output:

```
BFT (Adjacency Matrix):
0 1 2 3
DFT (Adjacency Matrix):
0 1 3 2
```

Adjacency Lists:

```
from collections import deque
graph_list = {
    0: [1, 2],
    1: [0, 3],
    2: [0, 3],
    3: [1, 2]
}

def bft_list(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        v = queue.popleft()
        print(v, end=' ')
        for neighbor in graph[v]:
            if neighbor not in visited:
                queue.append(neighbor)
                visited.add(neighbor)
```

```
def dft_list(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    print(start, end=' ')
    for neighbor in graph[start]:
        if neighbor not in visited:
            dft_list(graph, neighbor, visited)

print("\nBFT (Adjacency List):")
bft_list(graph_list, 0)
print("\nDFT (Adjacency List):")
dft_list(graph_list, 0)
```

output:

```
BFT (Adjacency List):
0 1 2 3
DFT (Adjacency List):
0 1 3 2
```

4.Write a program for finding the bi-connected components in a given graph

```
from collections import defaultdict
class Graph:
    def __init__(self, vertices):
        self.V = vertices
        self.graph = defaultdict(list)
        self.time = 0
    def add_edge(self, u, v):
        self.graph[u].append(v)
        self.graph[v].append(u)
    def _bcc_util(self, u, parent, disc, low, st, bcc_list):
        disc[u] = low[u] = self.time
        self.time += 1
        children = 0
        for v in self.graph[u]:
            if disc[v] == -1:
                children += 1
                st.append((u, v))
                self._bcc_util(v, u, disc, low, st, bcc_list)
                low[u] = min(low[u], low[v])
                if (parent == -1 and children > 1) or (parent != -1 and low[v] >= disc[u]):
                    bcc = []
                    while st[-1] != (u, v):
                        bcc.append(st.pop())
                    bcc.append(st.pop())
                    bcc_list.append(bcc)
            elif v != parent and disc[v] < disc[u]:
                low[u] = min(low[u], disc[v])
                st.append((u, v))
    def biconnected_components(self):
        disc = [-1] * self.V
        low = [-1] * self.V
        st = []
```

```
        bcc_list = []
        for i in range(self.V):
            if disc[i] == -1:
                self._bcc_util(i, -1, disc, low, st, bcc_list)
            if st:
                bcc = []
                while st:
                    bcc.append(st.pop())
                bcc_list.append(bcc)
        return bcc_list


g = Graph(5)
edges = [(0,1),(1,2),(2,0),(1,3),(3,4)]
for u, v in edges:
    g.add_edge(u, v)
bccs = g.biconnected_components()
for i, comp in enumerate(bccs, 1):
    print(f"Biconnected Component {i}: {comp}")
```

output:

Biconnected Component 1: [(3, 4)]

Biconnected Component 2: [(1, 3)]

Biconnected Component 3: [(2, 0), (1, 2), (0, 1)]

5.Implement Quick sort and Merge sort and observe the execution time for various input sizes (Average, Worst and Best cases).

```
import random
import time
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def measure_time(sort_func, arr):
    start = time.time()
```

```
    sort_func(arr)
    end = time.time()
    return end - start
sizes = [1000, 5000, 10000]
cases = ["Average", "Best", "Worst"]
for size in sizes:
    print(f"\nInput size: {size}")
    arr_avg = [random.randint(0, size) for _ in range(size)]
    arr_best = sorted(arr_avg)
    arr_worst = sorted(arr_avg, reverse=True)
    for case, arr in zip(cases, [arr_avg, arr_best, arr_worst]):
        time_quick = measure_time(quick_sort, arr.copy())
        time_merge = measure_time(merge_sort, arr.copy())
        print(f"{case} Case - Quick Sort: {time_quick:.5f}s, Merge Sort: {time_merge:.5f}s")
```

output:

```
Input size: 1000
Average Case - Quick Sort: 0.00229s, Merge Sort: 0.00594s
Best Case - Quick Sort: 0.00131s, Merge Sort: 0.00384s
Worst Case - Quick Sort: 0.00126s, Merge Sort: 0.00267s
Input size: 5000
Average Case - Quick Sort: 0.01103s, Merge Sort: 0.02673s
Best Case - Quick Sort: 0.00829s, Merge Sort: 0.01253s
Worst Case - Quick Sort: 0.01002s, Merge Sort: 0.01602s
Input size: 10000
Average Case - Quick Sort: 0.02439s, Merge Sort: 0.03977s
Best Case - Quick Sort: 0.02247s, Merge Sort: 0.03548s
Worst Case - Quick Sort: 0.01557s, Merge Sort: 0.02738s
```

6.Compare the performance of Single Source Shortest Paths using Greedy method when the graph is represented by adjacency matrix and adjacency lists.

```
import heapq
matrix = [
    [0, 2, 0, 6, 0],
    [2, 0, 3, 8, 5],
    [0, 3, 0, 0, 7],
    [6, 8, 0, 0, 9],
    [0, 5, 7, 9, 0]
]
alist = [
    [(1, 2), (3, 6)],
    [(0, 2), (2, 3), (4, 5)],
    [(1, 3), (4, 7)],
    [(0, 6), (1, 8), (4, 9)],
    [(1, 5), (2, 7), (3, 9)]
]
def dijkstra_matrix(graph, src):
    n = len(graph)
    dist = [float('inf')] * n
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        current_dist, u = heapq.heappop(pq)
        if current_dist > dist[u]:
            continue
        for v, weight in enumerate(graph[u]):
            if weight != 0:
                if dist[u] + weight < dist[v]:
                    dist[v] = dist[u] + weight
                    heapq.heappush(pq, (dist[v], v))
    return dist
def dijkstra_list(graph, src):
    n = len(graph)
```

```
    dist = [float('inf')] * n
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        current_dist, u = heapq.heappop(pq)
        if current_dist > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist
source_node = 0
print(f"shortest distamces from node {source_node}:")
dist_matrix = dijkstra_matrix(matrix, source_node)
print("Adjacency Matrix distances:", dist_matrix)
dist_list = dijkstra_list(alist, source_node)
print("Adjacency List distances:", dist_list)
```

output:

```
shortest distamces from node 0:
Adjacency Matrix distances: [0, 2, 5, 6, 7]
Adjacency List distances: [0, 2, 5, 6, 7]
```

7.Implement Job sequencing with deadlines using Greedy strategy.

```
class Job:
    def __init__(self, job_id, deadline, profit):
        self.id = job_id
        self.deadline = deadline
        self.profit = profit
def job_sequencing(jobs):
    # Step 1: Sort jobs by decreasing profit
    jobs.sort(key=lambda x: x.profit, reverse=True)
    n = max(job.deadline for job in jobs)
    result = [None] * n
    total_profit = 0
    for job in jobs:
        for slot in range(job.deadline - 1, -1, -1):
            if result[slot] is None:
                result[slot] = job.id
                total_profit += job.profit
                break
    scheduled_jobs = [job_id for job_id in result if job_id is not None]
    return scheduled_jobs, total_profit
if __name__ == "__main__":
    jobs = [
        Job('A', 7, 15),
        Job('B', 2, 20),
        Job('C', 5, 30),
        Job('D', 3, 18),
        Job('E', 4, 18),
        Job('F', 5, 10),
        Job('G', 2, 23),
        Job('H', 7, 16),
        Job('I', 3, 25)
    ]
    scheduled_jobs, total_profit = job_sequencing(jobs)
```

```
    print("☑ Scheduled Jobs:", scheduled_jobs)
    print("💰 Total Profit:", total_profit)
```

output:

☑ Scheduled Jobs: ['B', 'G', 'I', 'E', 'C', 'A', 'H']

💰 Total Profit: 147

8. Find Optimal solution for a Knap Sack Problem using Greedy strategy.

```
def fractional_knapsack(values, weights, capacity):
    ratio = [(v / w, v, w) for v, w in zip(values, weights)]
    ratio.sort(key=lambda x: x[0], reverse=True)
    total_value = 0.0
    for r, v, w in ratio:
        if capacity <= 0:
            break
        amount = min(w, capacity)
        total_value += (v / w) * amount
        capacity -= amount
    return total_value
values = [60, 100, 120]
weights = [10, 20, 30]
capacity = 50
max_value = fractional_knapsack(values, weights, capacity)
print(f"💰 Maximum value in Knapsack = {max_value}")
```

output:

```
Maximum value in Knapsack = 240.0
```

9.Write a program to solve 0/1 Knapsack problem Using Dynamic Programming.

```
def knapsack_dp(values, weights, capacity):
    n = len(values)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    dp[i - 1][w],
                    values[i - 1] + dp[i - 1][w - weights[i - 1]]
                )
            else:
                dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]
values = [60, 100, 120]
weights = [10, 20, 30]
W = 50
print("Maximum value in Knapsack =", knapsack_dp(values, weights, W))
```

output:

Maximum value in Knapsack = 220

10.Implement N-Queens Problem Using Backtracking.

```
def print_board(board):
    for row in board:
        print(" ".join('Q' if x else '.' for x in row))
    print()
def is_safe(board, row, col, n):
    # Check row on left
    for i in range(col):
        if board[row][i]:
            return False
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j]:
            return False
    for i, j in zip(range(row, n), range(col, -1, -1)):
        if board[i][j]:
            return False
    return True
def solve_nq_util(board, col, n):
    if col >= n:
        return True
    for i in range(n):
        if is_safe(board, i, col, n):
            board[i][col] = 1
            if solve_nq_util(board, col + 1, n):
                return True
            board[i][col] = 0
    return False
def solve_n_queens(n):
    board = [[0]*n for _ in range(n)]
    if not solve_nq_util(board, 0, n):
        print("No solution exists")
        return
    print_board(board)
```

n = 4

solve_n_queens(n)

output:

. . Q .

Q . . .

. . . Q

. Q . .

11.Use Backtracking strategy to solve 0/1 Knapsack problem.

```
def knapsack_backtrack(values, weights, capacity, n):
    if n == 0 or capacity == 0:
        return 0
    if weights[n - 1] > capacity:
        return knapsack_backtrack(values, weights, capacity, n - 1)
    else:
        return max(
            values[n - 1] + knapsack_backtrack(values, weights, capacity - weights[n - 1], n - 1),
            knapsack_backtrack(values, weights, capacity, n - 1)
        )
values = [60, 100, 120]
weights = [10, 20, 30]
W = 50
max_value = knapsack_backtrack(values, weights, W, len(values))
print("Maximum value in Knapsack =", max_value)
```

output:

Maximum value in Knapsack = 220

12.Implement Travelling Sales Person problem using Branch and Bound approach.

```
import itertools
def tsp(graph):
    n = len(graph)
    vertices = list(range(n))
    min_path = float('inf')
    best_order = []
    for perm in itertools.permutations(vertices[1:]):
        current_pathweight = 0
        k = 0
        for j in perm:
            current_pathweight += graph[k][j]
            k = j
        current_pathweight += graph[k][0]
        if current_pathweight < min_path:
            min_path = current_pathweight
            best_order = (0,) + perm + (0,)
    return min_path, best_order
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
min_cost, path = tsp(graph)
print("Minimum cost:", min_cost)
print("Path:", path)
```

output:

```
Minimum cost: 80
Path: (0, 1, 3, 2, 0)
```